

Fiche de révision — Machine Learning

Concours ANP — Cadre Data Scientist

Table des matières

1	Concepts ML de base	2
1.1	AI vs ML vs Deep Learning	2
1.2	Apprentissage supervisé vs non supervisé	2
1.3	Classification vs régression vs clustering	2
1.4	Training, testing, validation	2
1.5	Features et target	3
2	Algorithmes	3
2.1	Linear Regression	3
2.2	Logistic Regression	4
2.3	Decision Tree	5
2.4	Random Forest	7
2.5	XGBoost	8
2.6	K-Means	9
3	Évaluation ML	10
3.1	Classification — Confusion Matrix	10
3.2	Classification — Métriques	10
3.3	Régression — Métriques	11
3.4	Questions types	11
4	ML avancé (Jour 3)	12
4.1	Preprocessing	12
4.2	Feature engineering et feature selection	15
4.3	Data leakage	16
4.4	Imbalanced dataset et SMOTE	17
4.5	Cross-validation	18
4.6	Hyperparameter tuning	19
4.7	Gradient Boosting et Ensemble Learning	20
4.8	Questions ouvertes	21

1 Concepts ML de base

1.1 AI vs ML vs Deep Learning

- **Intelligence Artificielle (AI)** : le domaine le plus large — toute technique qui permet à une machine d'imiter un comportement humain intelligent (raisonnement, perception, décision). Inclut des règles écrites à la main (systèmes experts), pas forcément de l'apprentissage.
- **Machine Learning (ML)** : sous-ensemble de l'AI où la machine *apprend* un modèle à partir de données, au lieu de suivre des règles écrites explicitement. On lui montre des exemples, elle en déduit un pattern généralisable.
- **Deep Learning (DL)** : sous-ensemble du ML basé sur des réseaux de neurones à plusieurs couches (*deep* = profond). Excelle quand il y a beaucoup de données non structurées (images, texte, son) et peu de features déjà préparées à la main.

Hiérarchie à retenir : $AI \supset ML \supset \text{Deep Learning}$.

Idée clé à savoir formuler : le ML classique (Random Forest, XGBoost...) reste souvent préférable sur des données tabulaires (comme des données économiques ou portuaires) car plus interprétable et moins gourmand en données/calcul que le Deep Learning.

1.2 Apprentissage supervisé vs non supervisé

- **Supervised learning (apprentissage supervisé)** : on dispose de données déjà étiquetées, c'est-à-dire qu'on connaît la réponse attendue (*target*) pour chaque exemple historique. Le modèle apprend la relation entre les features et le target pour prédire ce target sur de nouvelles données.
 - Exemple : prédire si un navire sera en retard (oui/non) à partir de son historique.
- **Unsupervised learning (apprentissage non supervisé)** : les données n'ont *pas* de target connu. Le modèle cherche uniquement une structure ou des regroupements cachés dans les données.
 - Exemple : regrouper des navires ou des clients par profil similaire sans savoir à l'avance quels groupes existent.

1.3 Classification vs régression vs clustering

- **Classification** (supervisé) : le target est une *catégorie* discrète (ex. retard / pas de retard, fraude / pas fraude).
- **Régression** (supervisé) : le target est une valeur *continue* (ex. temps d'attente en heures, montant d'une facture).
- **Clustering** (non supervisé) : regrouper des observations similaires entre elles, sans target prédéfini (ex. segmentation de clients ou de navires).

1.4 Training, testing, validation

- **Training set (entraînement)** : les données sur lesquelles le modèle apprend ses paramètres.
- **Validation set** : sert à régler les hyperparamètres et comparer plusieurs modèles *sans* toucher aux données de test, pour éviter de biaiser l'évaluation finale.
- **Test set** : données jamais vues par le modèle pendant l'entraînement, utilisées une seule fois à la fin pour mesurer la performance réelle attendue en production.

Pourquoi séparer les trois : si on évalue le modèle sur les données qui ont servi à l'entraîner, on surestime sa performance réelle (voir *overfitting* plus loin).

1.5 Features et target

- **Features (variables explicatives)** : les colonnes/informations d'entrée utilisées par le modèle pour faire une prédiction (ex. tirant d'eau du navire, type de marchandise, heure d'arrivée).
- **Target (variable cible)** : ce qu'on cherche à prédire (ex. le temps d'attente du navire).

Question type à savoir répondre à l'oral :

« Quelle différence entre classification et régression, avec un exemple portuaire ? »

2 Algorithmes

2.1 Linear Regression

Principe : trouve la droite (ou l'hyperplan, avec plusieurs features) qui passe "au plus près" de tous les points observés, en minimisant la somme des écarts au carré entre la valeur prédite et la valeur réelle. Le modèle prédit une valeur *continue* comme une combinaison linéaire pondérée des features : $y = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_0$.

Formalisation mathématique — du simple au général

Étape 1 — l'équation avec 2 features seulement. Prenons le cas le plus simple, avec deux features x_1 et x_2 :

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Exemple concret : prédire le temps d'attente d'un navire (en heures) avec x_1 = tirant d'eau et x_2 = nombre de conteneurs à décharger. Une fois le modèle entraîné, $\theta_0, \theta_1, \theta_2$ sont simplement **3 nombres fixes** (par exemple $\theta_0 = 2, \theta_1 = 0.5, \theta_2 = 0.01$).

Étape 2 — les θ_j sont constants pour toutes les lignes. Une fois appris, $\theta_0, \theta_1, \theta_2$ ne changent *plus* : ce sont les mêmes 3 nombres réutilisés pour prédire n'importe quel nouveau navire. Ce qui change d'une ligne (d'un navire) à l'autre, ce sont uniquement les valeurs x_1, x_2 — jamais les θ :

Navire	x_1 (tirant d'eau)	x_2 (conteneurs)	$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$
A	8	200	$2 + 0.5(8) + 0.01(200) = 8$
B	12	500	$2 + 0.5(12) + 0.01(500) = 13$

Même $\theta_0, \theta_1, \theta_2$ dans les deux lignes — seuls x_1, x_2 changent d'un navire à l'autre. C'est cette propriété (paramètres partagés par toutes les observations) qui permet au modèle de *généraliser* à de nouveaux navires jamais vus.

Étape 3 — généralisation à p features. Pour une observation i avec un vecteur de features $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$, le modèle prédit :

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots + \theta_p x_{ip} = \boldsymbol{\theta}^\top \mathbf{x}_i$$

Le **double indice** x_{ij} (valeur de la feature j pour l'observation i) distingue bien les deux rôles : i indexe la *ligne* (le navire), j indexe la *feature* (tirant d'eau, nombre de conteneurs, ...). En revanche θ_j n'a **qu'un seul indice** : il ne dépend que de la feature j , jamais de la ligne i — exactement comme dans le tableau ci-dessus.

Remarque de notation : $\boldsymbol{\theta}$ et $\boldsymbol{\beta}$ désignent le même objet (le vecteur de coefficients) — $\boldsymbol{\theta}$ est la notation la plus courante côté Machine Learning, $\boldsymbol{\beta}$ vient de la tradition statistique.

On minimise uniquement par rapport à θ . Les données d'entraînement x_i, y_i sont *fixes* (ce sont des constantes du problème une fois le dataset choisi) — seuls les θ_j sont ajustés pour rendre l'erreur $J(\theta)$ la plus petite possible :

$$\hat{\theta} = \arg \min_{\theta} J(\theta)$$

C'est pour cela que la mise à jour du gradient (plus bas) s'écrit séparément pour chaque θ_j , avec sa propre dérivée partielle $\frac{\partial J}{\partial \theta_j}$: elle mesure combien J changerait si l'on bougeait légèrement *seulement* ce coefficient, tous les autres θ et toutes les données restant fixes. En pratique, tous les θ_j sont mis à jour simultanément à chaque itération.

Fonction de coût (Ordinary Least Squares). On cherche les paramètres qui minimisent la moyenne des carrés des résidus (*Mean Squared Error*) sur les m observations d'entraînement :

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Pourquoi le carré ? Le carré permet :

- d'éviter que les erreurs positives et négatives s'annulent ;
- de pénaliser fortement les grandes erreurs ;
- d'obtenir une fonction facilement optimisable par dérivation.

Le facteur 2 (qui apparaît dans certaines écritures de J , par ex. $\frac{1}{2m} \sum (\hat{y}_i - y_i)^2$) est principalement une convention mathématique qui facilite la dérivée : il s'annule avec le 2 provenant de la dérivée du carré.

Pourquoi m ? On veut calculer une moyenne des erreurs — diviser par m (le nombre d'observations) rend le coût comparable d'un dataset à l'autre, indépendamment du nombre d'exemples.

Lien avec Gradient Descent. On utilise souvent la descente de gradient pour trouver les paramètres θ qui minimisent cette fonction :

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

où α est le *learning rate*.

Hypothèses statistiques du modèle (utile si on te pousse sur la théorie) : linéarité de la relation, indépendance des résidus, homoscedasticité (variance des résidus constante), normalité des résidus, absence de multicollinéarité forte.

- **Type de problème** : régression (target continu).
- **Cas d'usage** : prédire le temps d'attente d'un navire en heures, prédire un montant, une quantité, un prix.
- **Limites** :
 - Suppose une relation *linéaire* entre features et target — capte mal les relations complexes/non linéaires.
 - Sensible aux *outliers* (valeurs extrêmes qui tirent la droite).
 - Sensible à la *colinéarité* entre features (deux features très corrélées entre elles perturbent l'interprétation des coefficients).
- **Avantage clé à mentionner** : très interprétable — chaque coefficient θ_j indique l'effet direct et quantifiable de la feature x_j sur le target, ce qui est précieux pour justifier une décision auprès d'une Direction.

2.2 Logistic Regression

Principe : malgré son nom, c'est un modèle de *classification*, pas de régression. Il prédit une *probabilité* (entre 0 et 1) d'appartenir à une classe, en appliquant une fonction sigmoïde à une combinaison linéaire des features. On fixe ensuite un seuil (souvent 0,5) pour décider de la classe finale : si probabilité $\geq 0,5 \rightarrow$ classe 1, sinon classe 0.

Formalisation mathématique

Fonction sigmoïde (logistique). On combine linéairement les features comme en régression linéaire, $z_i = \beta^\top \mathbf{x}_i + \beta_0$, puis on écrase ce score réel dans l'intervalle $[0, 1]$ via :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \Rightarrow \quad P(y_i = 1 \mid \mathbf{x}_i) = \hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-(\beta^\top \mathbf{x}_i + \beta_0)}}$$

Log-odds (logit). La régression logistique modélise en fait le logarithme du rapport de cotes comme une fonction linéaire des features, ce qui justifie le nom "logistic" :

$$\text{logit}(\hat{p}_i) = \ln\left(\frac{\hat{p}_i}{1 - \hat{p}_i}\right) = \beta^\top \mathbf{x}_i + \beta_0$$

Un coefficient β_j s'interprète donc comme l'effet de x_j sur le *log-odds* : e^{β_j} est le facteur multiplicatif appliqué au rapport de cotes quand x_j augmente de 1 unité.

Fonction de coût (Binary Cross-Entropy / Log-Loss). On ne peut pas utiliser la somme des carrés (non convexe ici) : on maximise la vraisemblance des observations, ce qui revient à minimiser la *log-loss* sur les n observations ($y_i \in \{0, 1\}$) :

$$J(\beta, \beta_0) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \right]$$

Cette fonction est convexe en β , ce qui garantit un minimum global unique.

Optimisation. Contrairement à la régression linéaire, il n'y a pas de solution analytique fermée : on optimise J par descente de gradient (ou variantes comme Newton-Raphson / IRLS) :

$$\beta_j \leftarrow \beta_j - \alpha \frac{\partial J}{\partial \beta_j} \quad \text{avec} \quad \frac{\partial J}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n x_{ij} (\hat{p}_i - y_i)$$

Remarque : cette dérivée a exactement la même forme que pour la régression linéaire (prédiction — réel, pondéré par la feature) — c'est une propriété élégante commune à tous les modèles linéaires généralisés (GLM) avec une fonction de lien canonique.

- **Type de problème** : classification (le plus souvent binaire ; extensible au multi-classe).
- **Cas d'usage** : prédire si un navire sera en retard (oui/non), détecter une anomalie ou une fraude, scoring d'un dossier (accepté/refusé).
- **Limites** :
 - Suppose que les classes sont *linéairement séparables* (une frontière de décision simple)
 - performe mal sur des frontières complexes.
 - Sensible aux features non standardisées et aux outliers.
 - Moins performante que Random Forest/XGBoost sur des données avec beaucoup d'interactions entre variables.
- **Avantage clé à mentionner** : tout comme la régression linéaire, elle reste interprétable (coefficients = effet sur le log-odds) et donne directement une probabilité utile pour prioriser des cas (ex. classer les navires par risque de retard décroissant), pas seulement une étiquette binaire.

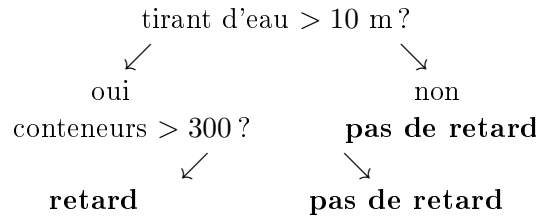
Point à ne pas confondre à l'oral : Linear Regression prédit un *nombre continu* ; Logistic Regression prédit une *probabilité*/catégorie. Les deux sont des modèles linéaires simples, souvent utilisés comme *baseline* avant de tester des modèles plus complexes (Random Forest, XGBoost).

2.3 Decision Tree

Étape 1 — l'intuition. Un arbre de décision pose une suite de questions simples de type "la feature x_j est-elle $\leq$ un seuil s ?" et découpe les données en deux groupes à chaque question,

jusqu'à obtenir des groupes finaux (*feuilles*) les plus "purs" possible (majoritairement une seule classe, ou une valeur de target homogène).

Exemple portuaire : "tirant d'eau > 10 m ?" → si oui, "nombre de conteneurs > 300 ?" → ... jusqu'à la feuille qui donne la prédiction (ex. "retard probable" / "pas de retard").



Étape 2 — comment choisir la meilleure question ? À chaque nœud, l'arbre teste toutes les features et tous les seuils possibles, et choisit celui qui rend les groupes résultants les plus purs. Pour mesurer l'impureté d'un groupe, deux critères principaux :

Indice de Gini (classification), pour un nœud contenant K classes de proportions $p_1, \dots, p_K$:

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

Gini = 0 si le nœud est parfaitement pur (une seule classe) ; Gini est maximal quand les classes sont mélangées à parts égales.

Entropie (alternative à Gini, issue de la théorie de l'information) :

$$\text{Entropie} = - \sum_{k=1}^K p_k \log_2(p_k)$$

Gain d'information. On choisit la question (feature + seuil) qui *maximise la réduction d'impureté* entre le nœud parent et ses deux enfants (gauche G , droit D), pondérée par la taille de chaque enfant :

$$\text{Gain} = \text{Impureté}_{\text{parent}} - \left(\frac{n_G}{n} \text{Impureté}_G + \frac{n_D}{n} \text{Impureté}_D \right)$$

où n est le nombre d'observations au nœud parent, n_G et n_D le nombre d'observations envoyées respectivement à gauche et à droite.

Cas de la régression : au lieu de Gini/entropie, on minimise la variance (ou la MSE) des valeurs de target dans chaque groupe — même logique de la fonction de coût que pour Linear Regression, appliquée localement à chaque nœud.

Arrêt de la construction. L'arbre s'arrête de grandir quand une condition est atteinte : profondeur maximale (**max_depth**), nombre minimal d'observations par feuille (**min_samples_leaf**), ou gain d'information devenu négligeable.

- **Type de problème** : classification et régression.
- **Cas d'usage** : règles de décision explicables (ex. scoring d'un dossier avec des critères métier lisibles), segmentation de navires par profil de risque.
- **Limites** :
 - **Overfitting très facile** si l'arbre est laissé libre de grandir (il peut isoler chaque observation dans sa propre feuille).
 - **Instable** : une petite variation dans les données d'entraînement peut produire un arbre très différent (forte variance).
 - Frontières de décision "en escalier" (perpendiculaires aux axes), donc moins naturelles que celles d'un modèle linéaire pour des relations lisses.
- **Avantage clé à mentionner** : très interprétable et visualisable (on peut littéralement dessiner l'arbre et le montrer à un métier non technique) ; ne nécessite ni scaling ni normalisation des features.

2.4 Random Forest

Étape 1 — le problème que ça résout. Un seul Decision Tree a une forte variance (instable, sensible à l'overfitting). L'idée du Random Forest : au lieu d'un seul arbre, en construire *beaucoup* (ex. 100, 500) et faire la **moyenne** de leurs prédictions (régression) ou un **vote majoritaire** (classification). Des erreurs individuelles d'arbres différents ont tendance à s'annuler, ce qui réduit la variance globale sans trop augmenter le biais.

Étape 2 — comment rendre les arbres différents entre eux ? Si tous les arbres étaient construits sur exactement les mêmes données avec les mêmes règles, ils seraient identiques et la moyenne n'apporterait rien. Random Forest introduit deux sources d'aléa (technique dite du *bagging* = *Bootstrap Aggregating*) :

- **Bootstrap sur les lignes** : chaque arbre est entraîné sur un échantillon tiré *avec remise* (même taille m que le dataset original, mais certaines lignes apparaissent plusieurs fois, d'autres pas du tout).
- **Sélection aléatoire de features** : à chaque nœud, l'arbre ne considère qu'un sous-ensemble aléatoire des features (souvent $\sqrt{p}$ pour la classification, $p/3$ pour la régression, avec p = nombre total de features) au lieu de toutes les tester — cela décorrèle davantage les arbres entre eux.

Formalisation de l'agrégation. Pour T arbres entraînés $f_1, \dots, f_T$:

$$\hat{y}_{\text{régression}} = \frac{1}{T} \sum_{t=1}^T f_t(\mathbf{x}) \qquad \hat{y}_{\text{classification}} = \text{mode}(f_1(\mathbf{x}), \dots, f_T(\mathbf{x}))$$

c'est-à-dire la moyenne des T prédictions numériques, ou la classe la plus votée parmi les T arbres.

Pourquoi ça marche (intuition de la réduction de variance). Si chaque arbre a une variance σ^2 et que les erreurs des arbres sont peu corrélées, la variance de la moyenne de T arbres tend vers σ^2/T — d'où l'intérêt de maximiser T (dans une limite raisonnable) et de décorrélérer les arbres (bootstrap + sélection aléatoire de features).

Feature importance (bonus utile à mentionner) : Random Forest fournit naturellement un score d'importance par feature, en mesurant de combien chaque feature réduit l'impureté (Gini/variance) en moyenne sur tous les arbres et tous les nœuds où elle est utilisée — utile pour expliquer un modèle à une Direction.

- **Type de problème** : classification et régression.
- **Cas d'usage** : la plupart des problèmes tabulaires "par défaut" (scoring, prévision, détection d'anomalie) — bon compromis performance/simplicité de mise en œuvre.
- **Limites** :
 - Moins interprétable qu'un seul Decision Tree (on ne peut plus "dessiner" la décision)
 - on s'appuie sur la feature importance en remplacement.
 - Plus coûteux en mémoire/temps de calcul qu'un modèle linéaire ou qu'un seul arbre (il faut stocker et faire tourner T arbres).
 - Peut rester moins performant que du Gradient Boosting/XGBoost sur des données très structurées, car les arbres sont construits indépendamment (pas de correction séquentielle des erreurs).
- **Avantage clé à mentionner** : robuste par défaut (peu de tuning nécessaire), gère bien les valeurs manquantes/outliers, réduit fortement l'overfitting par rapport à un arbre seul
 - c'est souvent le premier modèle "sérieux" à essayer après une baseline linéaire.

Question type à savoir répondre à l'oral : « *Pourquoi choisir Random Forest plutôt que Logistic Regression ?* » → Random Forest capte des relations non linéaires et des interactions complexes entre features sans les spécifier à la main, gère nativement les variables catégorielles et les valeurs manquantes, et est moins sensible aux outliers — au prix d'une interprétabilité directe des coefficients (compensée par la feature importance). Logistic Regression reste préférable quand

l'interprétabilité fine (impact exact et signé de chaque variable) est exigée, ou quand le volume de données est trop faible pour justifier un modèle complexe.

2.5 XGBoost

Étape 1 — boosting vs bagging. Random Forest construit ses T arbres *indépendamment* (en parallèle) puis moyenne leurs prédictions — c'est le *bagging*. Le *boosting* (dont XGBoost est la version la plus utilisée en pratique) construit ses arbres *séquentiellement* : chaque nouvel arbre est entraîné pour corriger les erreurs laissées par l'ensemble des arbres précédents, au lieu de repartir de zéro sur un échantillon aléatoire.

	Bagging (Random Forest)	Boosting (XGBoost)
Construction des arbres	indépendante (parallèle)	séquentielle
Objectif de chaque arbre	re-voter sur un échantillon	corriger l'erreur des précédents
Effet recherché	réduire la <i>variance</i>	réduire le <i>biais</i> (et la variance)

Étape 2 — l'intuition du boosting. On part d'une première prédiction simple $\hat{y}^{(0)}$ (par exemple la moyenne du target). À chaque étape t , on calcule le *résidu* (l'erreur restante) et on entraîne un nouvel arbre f_t pour prédire *ce résidu*, pas directement y :

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(\mathbf{x})$$

où η (*learning rate*, souvent petit : 0.01 à 0.3) contrôle la contribution de chaque nouvel arbre — avancer par petits pas évite de sur-corriger et de tomber dans l'overfitting.

Exemple portuaire : un premier arbre prédit un temps d'attente moyen grossier ; le deuxième arbre apprend spécifiquement sur quels navires ce premier arbre s'est le plus trompé, et corrige dans cette direction ; ainsi de suite.

Étape 3 — fonction objectif de XGBoost. XGBoost formalise l'ajout de chaque arbre comme la minimisation d'une fonction objectif qui combine une fonction de perte l (ex. MSE pour la régression, log-loss pour la classification) et un terme de régularisation Ω qui pénalise la complexité de l'arbre :

$$\text{Obj}^{(t)} = \sum_{i=1}^m l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad \text{avec} \quad \Omega(f_t) = \gamma T_{\text{feuilles}} + \frac{1}{2} \lambda \sum_k w_k^2$$

où T_{feuilles} est le nombre de feuilles de l'arbre f_t , w_k le poids (valeur de prédiction) de la feuille k , et γ, λ des hyperparamètres de régularisation. C'est ce terme Ω (absent du Gradient Boosting classique) qui donne au "X" (*eXtreme*) de XGBoost sa robustesse à l'overfitting.

Pourquoi la régularisation compte ici : sans elle, le boosting — qui cherche activement à corriger *chaque* erreur résiduelle — finirait par apprendre le bruit des données d'entraînement. Ω force les arbres à rester petits (peu de feuilles) et les poids de feuille modérés (via λ), un peu comme une régularisation L2 sur les coefficients d'une régression linéaire.

- **Type de problème** : classification et régression (et ranking).
- **Cas d'usage** : quasiment la référence actuelle sur données tabulaires en compétition/production — scoring fin, prévision, détection de fraude, quand on veut pousser la performance au-delà d'un Random Forest.
- **Limites** :
 - Plus d'hyperparamètres à régler (`learning_rate`, `max_depth`, `n_estimators`, γ , λ ...)
 - nécessite du tuning (grid search / cross-validation) pour être performant.
 - Entraînement séquentiel donc moins parallélisable que Random Forest (bien que XGBoost parallélise la construction *interne* de chaque arbre).

- Plus sensible au bruit/outliers que Random Forest si mal régularisé, puisqu’il cherche justement à corriger les erreurs résiduelles.
- **Avantage clé à mentionner** : très souvent la meilleure performance brute sur données tabulaires structurées, régularisation intégrée qui limite l’overfitting, gère nativement les valeurs manquantes (l’arbre apprend la meilleure direction par défaut pour les valeurs absentes).

2.6 K-Means

Étape 1 — l’intuition. K-Means est un algorithme de *clustering* (non supervisé) : il regroupe m observations en K groupes (*clusters*), sans connaître à l’avance les vraies étiquettes. L’idée : chaque cluster est résumé par un point central appelé *centroïde*, et chaque observation est assignée au cluster dont le centroïde est le plus proche.

Exemple portuaire : regrouper des navires en $K = 3$ profils (petits navires rapides, porte-conteneurs moyens, tankers volumineux) à partir de features comme le tonnage et le temps de manutention — sans savoir à l’avance combien de profils existent réellement ni comment les nommer.

Étape 2 — l’algorithme, pas à pas. Pour un nombre de clusters K fixé à l’avance :

1. **Initialisation** : choisir K centroïdes de départ (souvent K points tirés au hasard parmi les observations).
2. **Assignment** : affecter chaque observation $\mathbf{x}_i$ au centroïde μ_k le plus proche (distance euclidienne la plus courante) :

$$c_i = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \mu_k\|^2$$

3. **Mise à jour** : recalculer chaque centroïde comme la *moyenne* des observations qui lui sont assignées :

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

4. **Répéter** les étapes 2 et 3 jusqu’à convergence (les assignations ne changent plus, ou nombre maximal d’itérations atteint).

Étape 3 — que minimise K-Means ? L’algorithme minimise l’inertie intra-cluster (*Within-Cluster Sum of Squares*), c’est-à-dire la somme des distances au carré entre chaque point et le centroïde de son cluster :

$$J = \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \mu_k\|^2$$

Chaque étape (assignment puis mise à jour) fait strictement décroître (ou stagner) J — l’algorithme converge donc toujours, mais pas nécessairement vers le minimum *global* (il peut rester bloqué dans un minimum local selon l’initialisation).

Comment choisir K ? K est un hyperparamètre fixé *avant* l’entraînement, pas appris par l’algorithme. Deux méthodes classiques :

- **Méthode du coude (elbow method)** : tracer J en fonction de K ; J décroît toujours quand K augmente, mais on choisit le K où la décroissance ralentit nettement (le "coude" de la courbe).
- **Score de silhouette** : mesure à quel point chaque point est proche de son propre cluster comparé aux autres clusters (valeur entre -1 et 1 , plus c’est proche de 1 , mieux les clusters sont séparés).

Limite d’initialisation et parade. Comme le résultat dépend de l’initialisation, on relance généralement K-Means plusieurs fois avec des initialisations différentes et on garde la solution avec le J le plus bas (c’est ce que fait **k-means++** par défaut dans scikit-learn, qui choisit intelligemment les centroïdes de départ pour accélérer la convergence).

- **Type de problème** : clustering (non supervisé).
- **Cas d'usage** : segmentation de clients/navires/opérateurs portuaires par profil, détection de groupes homogènes pour adapter une politique tarifaire ou une allocation de ressources.
- **Limites** :
 - Il faut choisir K à l'avance (pas toujours évident métier).
 - Suppose des clusters de forme *sphérique* et de taille comparable (distance euclidienne)
 - performe mal sur des clusters de formes complexes ou de densités très différentes.
 - Sensible aux *outliers* (un point extrême peut fortement déplacer un centroïde) et à l'*échelle des features* (nécessite un scaling préalable, contrairement aux arbres).
- **Avantage clé à mentionner** : simple, rapide, facile à interpréter (chaque cluster est résumé par son centroïde) — bon premier réflexe pour explorer une structure cachée dans des données non étiquetées avant d'aller plus loin.

3 Évaluation ML

3.1 Classification — Confusion Matrix

Principe. Pour un problème de classification binaire, on compare la classe *prédite* par le modèle à la classe *réelle* pour chaque observation du test set. Il n'existe que 4 cas possibles, résumés dans la matrice de confusion :

		Réel	
		Positif	Négatif
Prédit	Positif	TP	FP
	Négatif	FN	TN

- **TP (Vrai Positif)** : le modèle prédit positif, et c'est effectivement positif. *Ex. : le modèle prédit "navire en retard", et il l'est vraiment.*
- **TN (Vrai Négatif)** : le modèle prédit négatif, et c'est effectivement négatif. *Ex. : le modèle prédit "pas de retard", et il n'y en a effectivement pas.*
- **FP (Faux Positif)** — "fausse alerte" : le modèle prédit positif, mais c'est en réalité négatif. *Ex. : le modèle annonce un retard qui n'arrive pas.*
- **FN (Faux Négatif)** — "occasion manquée" : le modèle prédit négatif, mais c'est en réalité positif. *Ex. : le modèle ne voit rien venir, mais le navire est bien en retard.*

Astuce pour ne jamais confondre à l'oral : le 2^e mot (Positif/Négatif) est toujours ce que le modèle a *prédit* ; le 1^{er} mot (Vrai/Faux) indique si cette prédiction était *correcte* ou non.

3.2 Classification — Métriques

Accuracy — proportion de prédictions correctes sur l'ensemble des prédictions :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Limite majeure : trompeuse sur un dataset déséquilibré. Si 95% des navires ne sont pas en retard, un modèle qui prédit toujours "pas de retard" obtient 95% d'accuracy sans être utile.

Precision — parmi les cas prédits positifs, combien le sont vraiment (mesure la fiabilité des alertes) :

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (rappel/sensibilité) — parmi les cas réellement positifs, combien le modèle en a détecté (mesure la capacité à ne rien manquer) :

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score — moyenne harmonique de Precision et Recall, utile quand on veut un seul chiffre résumant le compromis entre les deux :

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Pourquoi une moyenne harmonique et pas arithmétique ? Elle pénalise fortement le déséquilibre : si Precision = 1 et Recall = 0, la moyenne arithmétique donnerait 0.5 (trompeur), alors que le F1 donne 0 (reflète bien qu'un des deux aspects est totalement raté).

ROC-AUC. La courbe ROC trace le taux de vrais positifs ($\text{Recall} = TP/(TP+FN)$) contre le taux de faux positifs ($FPR = FP/(FP+TN)$) pour *tous les seuils de décision possibles* (pas seulement 0,5). L'AUC (*Area Under the Curve*) résume cette courbe en un seul nombre entre 0 et 1 :

- AUC = 1 : séparation parfaite entre les deux classes.
- AUC = 0.5 : le modèle n'est pas meilleur qu'un tirage aléatoire.

Intérêt par rapport à Accuracy/Precision/Recall : l'AUC évalue le modèle indépendamment du choix d'un seuil précis — utile pour comparer des modèles avant de décider où placer le seuil de décision selon le contexte métier.

3.3 Régression — Métriques

MAE (Mean Absolute Error) — moyenne des erreurs en valeur absolue, dans la même unité que le target (facile à interpréter) :

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

MSE (Mean Squared Error) — moyenne des erreurs au carré (pénalise fortement les grosses erreurs, cf. section Linear Regression pour la justification du carré) :

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

RMSE (Root Mean Squared Error) — racine carrée du MSE, ce qui la ramène dans la *même unité* que le target (contrairement au MSE) tout en gardant la pénalisation forte des grandes erreurs :

$$\text{RMSE} = \sqrt{\text{MSE}}$$

R^2 (coefficient de détermination) — proportion de la variance du target expliquée par le modèle, comparée à un modèle "naïf" qui prédirait toujours la moyenne $\bar{y}$:

$$R^2 = 1 - \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2}$$

- $R^2 = 1$: le modèle prédit parfaitement.
- $R^2 = 0$: le modèle ne fait pas mieux que prédire la moyenne.
- $R^2 < 0$: le modèle fait *pire* que la moyenne (possible sur un test set, signe d'un modèle mal ajusté).

3.4 Questions types

Precision vs Recall — quand privilégier laquelle ? Le choix dépend du coût relatif d'un FP par rapport à un FN :

- **Privilégier Precision** quand un FP coûte cher (fausse alerte coûteuse à traiter) — ex. mobiliser une équipe portuaire pour un retard qui n'arrive finalement pas.

- **Privilégier Recall** quand un FN coûte cher (rater un cas positif est grave) — ex. ne pas détecter un navire réellement en retard et donc ne pas anticiper l'engorgement du quai.

Il y a toujours un compromis : augmenter le seuil de décision augmente la Precision mais baisse le Recall, et inversement — d'où l'intérêt du F1-score ou de l'AUC pour arbitrer.

Qu'est-ce que l'overfitting ? Le modèle apprend "par cœur" les particularités (et le bruit) des données d'entraînement au lieu d'apprendre la tendance générale — il obtient une excellente performance sur le train set mais une performance nettement plus faible sur des données jamais vues (test set). L'inverse, l'*underfitting*, correspond à un modèle trop simple qui ne capte même pas la tendance générale (mauvaise performance sur train *et* test).

Comment le détecter ? Comparer la performance sur le train set et sur le test/validation set : un grand écart (très bon sur train, nettement moins bon sur test) est le signe caractéristique de l'overfitting.

Comment le réduire ?

- Plus de données d'entraînement (si possible).
- Régularisation (L1/L2 pour les modèles linéaires ; γ, λ pour XGBoost ; profondeur maximale/nombre minimal d'observations par feuille pour les arbres).
- Réduire la complexité du modèle (moins de features, arbre moins profond, moins d'arbres).
- Cross-validation pour détecter l'overfitting de façon fiable pendant le choix des hyperparamètres.
- Early stopping (arrêter l'entraînement dès que la performance sur validation cesse de s'améliorer, utile pour XGBoost et le boosting en général).
- Dropout/data augmentation (plutôt côté Deep Learning, à mentionner si la question dérive vers les réseaux de neurones).

4 ML avancé (Jour 3)

4.1 Preprocessing

Pourquoi le preprocessing est incontournable. Un modèle ne peut apprendre que sur des données propres, complètes et sur une échelle cohérente. En pratique, sur un projet réel (ex. données portuaires venant de plusieurs systèmes), le preprocessing prend souvent plus de temps que le choix de l'algorithme lui-même.

Missing values (valeurs manquantes)

Étape 1 — pourquoi ça arrive. Capteur défaillant, champ non renseigné dans un formulaire, fusion de plusieurs sources (JOIN) qui ne coïncident pas parfaitement. *Exemple portuaire* : le tirant d'eau n'est pas toujours transmis par tous les navires.

Étape 2 — détecter. En pandas : `df.isna().sum()` ou `df.info()` pour repérer les colonnes incomplètes et quantifier le taux de valeurs manquantes par colonne.

Étape 3 — stratégies de traitement.

- **Suppression (dropna)** : supprimer la ligne ou la colonne. Acceptable seulement si le taux de valeurs manquantes est faible *et* que les données manquent de façon aléatoire (pas de perte d'information systématique).
- **Imputation simple** : remplacer par la moyenne/médiane (variable numérique — médiane préférable si outliers présents) ou par le mode (variable catégorielle).
- **Imputation par groupe** : ex. remplacer le tirant d'eau manquant par la médiane *du même type de navire*, plus fine qu'une médiane globale.
- **Imputation par modèle** : prédire la valeur manquante à partir des autres features (ex. KNN imputer, régression) — plus précis mais plus coûteux.

- **Indicateur de manquant** : ajouter une colonne binaire `is_missing` avant d'imputer, pour ne pas perdre l'information que la valeur était absente (parfois ce fait lui-même est prédictif).

Piège à mentionner à l'oral : ne jamais calculer la moyenne/médiane d'imputation sur *tout* le dataset avant le split train/test — il faut la calculer uniquement sur le train, puis l'appliquer au test (sinon fuite d'information, voir *Data leakage* plus loin).

Outliers (valeurs aberrantes)

Étape 1 — détecter.

- **Méthode IQR (interquartile range)** : un point est outlier s'il sort de l'intervalle $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ avec $IQR = Q_3 - Q_1$, où **Q1 (1^{er} quartile)** est la valeur en dessous de laquelle se trouvent 25 % des données, et **Q3 (3^e quartile)** celle en dessous de laquelle se trouvent 75 % des données. Visualisable avec un *boxplot*.
- **Z-score** : un point est outlier si $|z| = \left| \frac{x-\mu}{\sigma} \right| > 3$ environ — suppose une distribution proche de la normale.
- **Visuel** : boxplot, scatter plot, histogramme.

Étape 2 — traiter. Ne jamais supprimer un outlier automatiquement sans comprendre sa cause :

- **Erreur de saisie** (ex. tirant d'eau de 999 m) → corriger ou supprimer.
- **Valeur réelle mais extrême** (ex. un très grand porte-conteneurs) → à garder, éventuellement traiter avec un modèle robuste (arbres) plutôt qu'un modèle linéaire sensible aux outliers.
- **Capping/winsorizing** : plafonner la valeur aux bornes de l'IQR (ou à un percentile, ex. 1^{er}/99^e) plutôt que supprimer la ligne — garde l'observation sans laisser l'extrême dominer.
- **Transformation** (ex. log) pour réduire l'effet des valeurs extrêmes sur une distribution très asymétrique.

Encoding (variables catégorielles → numérique)

Pourquoi encoder. Un modèle ne comprend que des nombres. Une colonne texte comme `type_conteneur` (sec/frigo/citerne) doit donc être transformée en valeurs numériques avant l'entraînement — sauf pour les arbres (Random Forest, XGBoost), qui savent parfois gérer le catégoriel nativement.

1. Label Encoding. On remplace chaque catégorie par un entier : 0, 1, 2, ...

Danger : cela crée un ordre qui n'existe pas forcément. Si `sec=0`, `frigo=1`, `citerne=2`, un modèle linéaire va comprendre que `citerne > frigo > sec`, comme s'il y avait une distance ou une hiérarchie entre ces catégories. Or "type de marchandise" n'a pas d'ordre naturel — c'est une variable *nominale*, pas *ordinaire*.

2. One-Hot Encoding. On crée une colonne binaire par catégorie.

Avant :

<code>type_conteneur</code>
sec
frigo
citerne
sec

Après One-Hot Encoding :

type_sec	type_frigo	type_citerne
1	0	0
0	1	0
0	0	1
1	0	0

Le **1** signifie : cette ligne appartient à cette catégorie. Le **0** signifie : cette ligne n'appartient pas à cette catégorie.

Chaque ligne n'a qu'un seul 1. Aucune colonne n'est "plus grande" qu'une autre — l'ordre artificiel du Label Encoding disparaît.

3. Pourquoi ça "augmente la dimensionnalité"

Imaginons une variable `type_conteneur` avec seulement 3 catégories. One-Hot → 3 colonnes.

Mais imaginons une variable `ville` avec 500 villes différentes. One-Hot Encoding va créer :

`ville_casablanca, ville_rabat, ville_tanger, ville_fes, ...`

→ 500 colonnes.

Donc une seule variable au départ devient 500 variables. C'est ce qu'on appelle **l'augmentation de la dimensionnalité** (*curse of dimensionality*). Conséquences concrètes : dataset beaucoup plus large et creux (*sparse*, majoritairement des 0), entraînement plus lent, risque accru d'overfitting (voir section overfitting) car le modèle a beaucoup plus de paramètres à apprendre pour la même quantité de données.

4. Ordinal Encoding. Même principe que le Label Encoding (chaque catégorie → un entier), mais réservé aux variables où l'ordre a vraiment un sens (ex. "faible"=0, "moyen"=1, "élevé"=2).

5. Target Encoding (plus avancé). On remplace chaque catégorie par la moyenne du target observée pour cette catégorie. Puissant, mais risqué : si la moyenne est calculée sur tout le dataset (train + test), on introduit un *data leakage*. Elle doit être calculée uniquement sur le train, souvent avec une validation croisée interne pour éviter le sur-apprentissage.

Règle pratique : Label/Ordinal Encoding pour les arbres (Random Forest, XGBoost — peu sensibles à un ordre arbitraire) ; One-Hot Encoding pour les modèles linéaires/distance (Logistic Regression, K-Means) sauf si trop de modalités, auquel cas on préfère le Target Encoding ou un regroupement des catégories rares avant encodage.

Scaling, normalization, standardization

Pourquoi c'est nécessaire. Si une feature va de 0 à 500 000 (ex. tonnage) et une autre de 0 à 20 (ex. tirant d'eau), les modèles basés sur une distance ou un gradient (Logistic Regression, K-Means, réseaux de neurones) donneront un poids disproportionné à la feature de plus grande échelle — pas parce qu'elle est plus importante, mais simplement parce que ses valeurs sont numériquement plus grandes.

— **Normalization (Min-Max Scaling)** : ramène chaque feature dans $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Sensible aux outliers (un seul point extrême écrase l'échelle de tous les autres). Utile quand on connaît les bornes réelles et qu'on veut un intervalle fixe (ex. entrée d'un réseau de neurones).

— **Standardization (Z-score scaling)** : centre-réduit la feature (moyenne 0, écart-type 1) :

$$x' = \frac{x - \mu}{\sigma}$$

Moins sensible aux outliers que le Min-Max, et généralement préférée par défaut pour les modèles linéaires et K-Means.

Quels modèles en ont besoin ?

- **Nécessaire** : Logistic/Linear Regression (pour un gradient descent stable et des coefficients comparables), K-Means/KNN (distance euclidienne), PCA (sensible à la variance de chaque axe), réseaux de neurones.
- **Inutile** : Decision Tree, Random Forest, XGBoost — les arbres découpent feature par feature sur des seuils, l'échelle absolue ne change pas l'ordre des observations donc ne change pas le découpage.

Piège data leakage (même logique que pour l'imputation) : **fit** le scaler (calcul de μ, σ ou $x_{\min}, x_{\max}$) uniquement sur le train set, puis **transform** le train *et* le test avec ces mêmes paramètres — ne jamais re-fit sur le test.

Question type à savoir répondre à l'oral : « Pourquoi le scaling est nécessaire pour K-Means mais pas pour Random Forest ? » → K-Means utilise une distance euclidienne entre points, donc une feature à grande échelle domine artificiellement le calcul de distance ; Random Forest ne compare jamais la magnitude absolue entre deux features différentes, il teste juste des seuils indépendamment sur chaque feature.

4.2 Feature engineering et feature selection

Distinction à ne pas confondre à l'oral : le *feature engineering* **crée** de nouvelles features à partir des données brutes ; la *feature selection* **réduit** l'ensemble de features en ne gardant que les plus utiles. Les deux visent le même but — améliorer la performance et la robustesse du modèle — mais dans des directions opposées (l'un ajoute, l'autre retire).

Feature engineering

Principe. Les données brutes ne sont pas toujours dans la forme la plus utile pour un modèle. Le feature engineering consiste à transformer ou combiner les colonnes existantes pour créer de nouvelles variables plus informatives.

Techniques courantes :

- **Extraction depuis une date/heure** : à partir d'un **timestamp** d'arrivée d'un navire, créer **jour_semaine**, **heure**, **est_weekend**, **mois** — un modèle ne peut pas exploiter un timestamp brut, mais peut apprendre "les arrivées le lundi matin ont plus souvent du retard".
- **Combinaison de features existantes** : ratio, différence, produit. *Exemple portuaire* : $\text{charge_par_metre} = \text{tonnage} / \text{longueur_quai}$, une variable dérivée souvent plus prédictive que les deux variables d'origine prises séparément.
- **Agrégations** : pour un navire donné, calculer le nombre moyen d'escales du même armateur sur les 30 derniers jours, ou le temps d'attente moyen historique sur ce quai — injecte du contexte que la ligne seule ne contient pas.
- **Binning (discrétisation)** : transformer une variable continue en catégories (ex. tirant d'eau → "faible"/"moyen"/"élevé") — utile si la relation avec le target n'est pas linéaire mais change par paliers.
- **Transformation mathématique** : log, racine carrée — réduit l'asymétrie d'une distribution (ex. tonnage très étalé vers les grandes valeurs) et atténue l'effet des outliers.

Idée clé : un bon feature engineering, guidé par la connaissance métier (ici, portuaire), améliore souvent plus la performance qu'un algorithme plus sophistiqué — d'où l'expression "*garbage in, garbage out*" : un modèle puissant sur des features pauvres reste un modèle pauvre.

Feature selection

Principe. Toutes les features disponibles ne sont pas utiles ; certaines sont redondantes, bruitées, ou non liées au target. En garder trop nuit au modèle : risque accru d'overfitting, temps

d'entraînement plus long, et interprétabilité réduite (voir *curse of dimensionality* évoquée pour le One-Hot Encoding).

Trois familles de méthodes :

- **Filter methods** (indépendantes du modèle) : on évalue chaque feature seule par une mesure statistique — corrélation avec le target, test du χ^2 pour des variables catégorielles — puis on garde les features au-dessus d'un seuil. Rapide, mais ignore les interactions entre features.
- **Wrapper methods** : on teste des sous-ensembles de features en entraînant réellement le modèle sur chacun et en comparant la performance. *Recursive Feature Elimination (RFE)* : on entraîne le modèle, on retire la feature la moins importante, on répète jusqu'à atteindre le nombre de features souhaité. Plus précis que les filter methods, mais beaucoup plus coûteux en calcul.
- **Embedded methods** : la sélection est intégrée directement dans l'entraînement du modèle. Ex. la régularisation **L1 (Lasso)** pousse certains coefficients exactement à 0, éliminant de fait les features correspondantes ; la *feature importance* de Random Forest/XGBoost (vue en section Algorithmes) permet aussi d'écarter les features les moins utiles après coup.

Question type à savoir répondre à l'oral : « *Pourquoi ne pas garder toutes les features disponibles ?* » → plus de features n'améliore pas forcément la performance : au-delà d'un certain point, le modèle apprend du bruit plutôt qu'un vrai signal (overfitting), l'entraînement devient plus lent, et le modèle plus difficile à interpréter et à expliquer à un métier non technique.

4.3 Data leakage

Définition. Il y a *data leakage* (fuite de données) quand une information qui ne serait **pas disponible au moment réel de la prédiction** se retrouve, directement ou indirectement, dans les données utilisées pour entraîner le modèle. Le modèle "triche" sans qu'on s'en rende compte : il obtient une excellente performance en test... et s'effondre en production, où cette information n'existe plus.

Pourquoi c'est dangereux. Contrairement à l'overfitting classique (écart visible entre score train et score test), le data leakage peut donner un **excellent score sur le test set lui-même** — car le test set est souvent contaminé par la même fuite. Le problème n'apparaît qu'une fois le modèle déployé, ce qui le rend particulièrement traître à détecter avant la mise en production.

Deux grandes familles de leakage

1. Target leakage (fuite via une feature). Une feature contient, de façon déguisée, de l'information sur le target — parce qu'elle n'est en réalité connue qu'*après* l'événement qu'on cherche à prédire.

Exemple portuaire : prédire si un navire sera "en retard" en utilisant comme feature `duree_reelle_manutention`. Problème : cette durée n'est connue qu'*une fois* le navire déchargé, donc après qu'on connaît déjà le retard. Le modèle apprend une relation qui n'existera jamais au moment où la prédiction doit réellement être faite (à l'arrivée du navire, avant manutention).

Autre exemple classique : prédire un défaut de paiement en utilisant une colonne `date_de_relance_impayé` — cette colonne n'existe que si le défaut a déjà eu lieu.

2. Train-test contamination (fuite via le split). Une information du test set "fuite" vers le train set pendant la préparation des données, avant ou pendant l'entraînement. Le document a déjà croisé plusieurs formes concrètes de ce type de fuite :

- **Imputation** : calculer la moyenne/médiane de remplacement sur *tout* le dataset avant le split, au lieu de la calculer uniquement sur le train (voir *Missing values*).
- **Scaling** : faire un **fit** du scaler (min/max, μ, σ) sur tout le dataset au lieu du train seul (voir *Scaling, normalization, standardization*).

- **Target Encoding** : calculer la moyenne du target par catégorie sur l'ensemble des données au lieu du train uniquement (voir *Encoding*).
- **Doublons** : la même observation (ou une observation très proche, ex. deux escales du même navire à quelques heures d'écart) présente à la fois dans le train et dans le test — le modèle "reconnait" l'exemple au lieu de généraliser.
- **Fuite temporelle** : pour des données chronologiques (ex. séries d'arrivées de navires), faire un split aléatoire au lieu d'un split *par date* — le modèle s'entraîne alors sur des données futures pour prédire le passé, une situation impossible en production.

Règle générale pour l'éviter. Toujours faire le split train/test **en premier**, avant toute étape de preprocessing qui "apprend" quelque chose des données (moyenne, min/max, encodage). Ensuite : **fit** uniquement sur le train, puis **transform** sur le train *et* le test avec les mêmes paramètres appris. En pratique, un **Pipeline** scikit-learn combiné à la cross-validation (section suivante) automatise cette discipline et réduit fortement le risque d'erreur humaine.

Question type à savoir répondre à l'oral : « *Comment détecter un data leakage ?* »
→ une performance test anormalement élevée (proche de 100%) est le premier signal d'alerte ; on vérifie ensuite si une feature très fortement corrélée au target aurait pu, dans les faits, être connue *avant* l'événement à prédire, et si toutes les étapes de preprocessing ont bien été calculées uniquement sur le train.

4.4 Imbalanced dataset et SMOTE

Le problème. Un dataset est *déséquilibré* (*imbalanced*) quand une classe est très majoritaire par rapport à l'autre. *Exemple portuaire* : sur l'historique des navires, seulement 5 % sont réellement "en retard" — 95 % "à l'heure".

Pourquoi l'Accuracy trompe ici. Un modèle qui prédit toujours "pas de retard", sans rien avoir appris, obtient 95 % d'Accuracy — excellent chiffre, modèle totalement inutile (il ne détecte jamais un seul retard). C'est exactement le piège déjà signalé en section *Évaluation ML* : sur un dataset déséquilibré, il faut regarder Precision/Recall/F1/ROC-AUC (voire la courbe Precision-Recall, plus informative que la ROC quand la classe positive est rare), jamais l'Accuracy seule.

Stratégies de traitement

1. Au niveau des données (resampling)

- **Undersampling** : réduire la classe majoritaire en supprimant aléatoirement des observations, pour rééquilibrer les proportions. Simple, mais fait perdre de l'information (des navires "à l'heure" informatifs sont jetés).
- **Oversampling** : dupliquer des observations de la classe minoritaire. Simple, mais risque de faire *overfitter* sur ces quelques exemples dupliqués (le modèle les voit plusieurs fois à l'identique).
- **SMOTE (Synthetic Minority Oversampling Technique)** : au lieu de dupliquer, on *génère* de nouvelles observations synthétiques de la classe minoritaire. Pour un point minoritaire donné, SMOTE cherche ses k plus proches voisins (même classe), puis crée un nouveau point sur le segment qui relie ce point à un voisin choisi au hasard :

$$x_{\text{nouveau}} = x_i + \lambda \cdot (x_{\text{voisin}} - x_i), \quad \lambda \sim \mathcal{U}(0, 1)$$

Le nouveau point est donc une interpolation plausible entre deux exemples minoritaires réels, pas une simple copie — ce qui limite le risque d'overfitting comparé à l'oversampling naïf.

2. Au niveau du modèle (sans toucher aux données)

- **Class weights** : pénaliser davantage les erreurs sur la classe minoritaire dans la fonction de coût (ex. `class_weight='balanced'` en scikit-learn) — le modèle "paie plus cher" un faux négatif sur un navire réellement en retard.

- **Ajuster le seuil de décision** : au lieu du seuil par défaut de 0,5 sur la probabilité prédite, l'abaisser pour privilégier le Recall (voir section *Questions types* sur le compromis Precision/Recall).
- **Choix de métrique adapté** lors du tuning (F1, ROC-AUC ou PR-AUC plutôt qu'Accuracy) pour ne pas optimiser le mauvais objectif.

Piège à mentionner à l'oral : appliquer SMOTE (ou tout resampling) **avant** le split train/test — ou avant chaque pli de cross-validation — crée un *data leakage* : des points synthétiques dérivés d'observations du train se retrouvent dans le test, ce qui gonfle artificiellement la performance mesurée. Le resampling doit être appliqué *uniquement* sur le train, jamais sur le test (le test doit rester représentatif de la réalité, donc déséquilibré).

Question type à savoir répondre à l'oral : « *Que faire si les classes sont déséquilibrées ?* »
→ ne pas se fier à l'Accuracy ; utiliser SMOTE ou class weights pour rééquilibrer l'apprentissage ; choisir une métrique adaptée (F1, ROC-AUC, PR-AUC) ; toujours resampler après le split, sur le train uniquement.

4.5 Cross-validation

Le problème que ça résout. Un seul split train/test donne une mesure de performance qui dépend du hasard de ce découpage précis — une observation "chanceuse" ou "malchanceuse" dans le test set peut faire varier le score sans que le modèle ait réellement changé. La cross-validation (validation croisée) donne une estimation plus fiable et plus stable de la performance réelle.

K-Fold Cross-Validation — principe pas à pas

1. On découpe le dataset d'entraînement en K sous-ensembles (*folds*) de taille à peu près égale (typiquement $K = 5$ ou $K = 10$).
2. On répète K fois : à chaque itération, un fold différent sert de *validation*, et les $K - 1$ folds restants servent d'entraînement.
3. On calcule la métrique choisie (F1, RMSE, ...) à chaque itération, puis on moyenne les K scores obtenus — et on peut aussi regarder leur écart-type, qui indique la *stabilité* du modèle selon le découpage.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Itération 1	val	train	train	train	train
Itération 2	train	val	train	train	train
Itération 3	train	train	val	train	train
Itération 4	train	train	train	val	train
Itération 5	train	train	train	train	val

Pourquoi c'est mieux qu'un seul split : chaque observation sert exactement une fois de validation et $K - 1$ fois d'entraînement — la mesure de performance finale (moyenne des K scores) ne dépend plus d'un découpage particulier "chanceux", et son écart-type révèle si le modèle est stable ou très sensible aux données utilisées.

Usage principal : comparer des modèles ou choisir des hyperparamètres (voir section suivante) de façon fiable, *sans jamais toucher au test set final* — celui-ci reste réservé à l'évaluation tout à la fin, une seule fois.

Variantes à connaître

- **Stratified K-Fold** : conserve la même proportion de classes dans chaque fold que dans le dataset global. Indispensable sur un dataset déséquilibré (voir sous-section précédente)

- sinon un fold pourrait se retrouver avec très peu, voire aucun exemple de la classe minoritaire.
- **Time Series Split** : pour des données chronologiques (ex. arrivées de navires dans le temps), les folds doivent respecter l'ordre temporel — on valide toujours sur des données *postérieures* au train de ce pli, jamais l'inverse (sinon fuite temporelle, déjà vue en *Data leakage*).
- **Leave-One-Out** : cas extrême où K = nombre d'observations (chaque pli valide sur une seule ligne). Très coûteux, réservé aux petits datasets.

Piège à mentionner à l'oral : tout le preprocessing qui "apprend" des données (imputation, scaling, target encoding, SMOTE) doit être re-fit à l'intérieur de chaque pli, uniquement sur la portion train de ce pli — jamais une seule fois sur tout le dataset avant la cross-validation, sinon c'est à nouveau un data leakage. Un Pipeline scikit-learn combiné à `cross_val_score` gère cette discipline automatiquement.

4.6 Hyperparameter tuning

Paramètre vs hyperparamètre — distinction à ne pas confondre à l'oral. Un *paramètre* est appris automatiquement par le modèle pendant l'entraînement (ex. les θ_j de la régression linéaire). Un *hyperparamètre* est fixé **avant** l'entraînement, par le data scientist — le modèle ne l'apprend pas lui-même (ex. `max_depth` d'un arbre, K de K-Means, `learning_rate` de XGBoost).

Pourquoi régler les hyperparamètres. Les valeurs par défaut d'une librairie ne sont presque jamais optimales pour un dataset donné. Un `max_depth` trop grand fait overfitter un arbre ; un `learning_rate` trop élevé fait diverger XGBoost ; un K mal choisi donne des clusters K-Means peu pertinents.

Méthodes de recherche

- **Grid Search** : on définit une grille de valeurs possibles pour chaque hyperparamètre (ex. `max_depth` $\in \{3, 5, 7\}$, `n_estimators` $\in \{100, 300\}$), puis on teste **exhaustivement** toutes les combinaisons. Garantit de trouver la meilleure combinaison *dans la grille définie*, mais le nombre de combinaisons explose vite (*curse of dimensionality* à nouveau) : 6 combinaisons ici, mais bien plus avec davantage d'hyperparamètres/valeurs.
- **Random Search** : au lieu de tout tester, on tire aléatoirement un nombre fixé de combinaisons dans l'espace des hyperparamètres. Contre- intuitivement souvent aussi performant que Grid Search pour un budget de calcul donné, car certains hyperparamètres ont beaucoup plus d'impact que d'autres — Random Search explore mieux ces dimensions importantes sans perdre de temps sur une grille rigide.
- **Bayesian Optimization** (plus avancé) : utilise les résultats des essais précédents pour choisir intelligemment la prochaine combinaison à tester (au lieu du hasard ou d'une grille fixe), en modélisant la relation entre hyperparamètres et performance. Converge généralement plus vite que Grid/Random Search, au prix d'une mise en œuvre plus complexe.

Comment évaluer chaque combinaison : jamais sur le test set final — on utilise la *cross-validation* (sous-section précédente) sur le train set pour comparer les combinaisons de façon fiable, et le test set ne sert qu'une seule fois, à la toute fin, pour confirmer la performance de la meilleure combinaison retenue.

Piège à mentionner à l'oral : tuner les hyperparamètres en observant directement le score sur le test set revient à laisser le test set "contaminer" indirectement le choix du modèle — c'est une forme de *data leakage* (le test n'est alors plus une mesure honnête de la performance en production).

Question type à savoir répondre à l'oral : « *Grid Search ou Random Search ?* » → Grid Search si peu d'hyperparamètres avec peu de valeurs candidates (recherche exhaustive

abordable) ; Random Search dès que l'espace de recherche devient grand, pour un meilleur rapport performance/temps de calcul.

4.7 Gradient Boosting et Ensemble Learning

Ensemble Learning — l'idée générale. Plutôt que d'utiliser un seul modèle, on combine les prédictions de **plusieurs** modèles pour obtenir une prédiction finale plus robuste que chacun pris isolément. L'intuition : des modèles différents font des erreurs différentes ; en les combinant, ces erreurs ont tendance à se compenser plutôt qu'à s'additionner. Random Forest et XGBoost (section Algorithmes) sont déjà deux exemples d'Ensemble Learning — cette sous-section prend du recul et situe les trois grandes familles.

Les trois familles d'Ensemble Learning

	Principe	Exemple
Bagging	modèles indépendants, moyennés	Random Forest
Boosting	modèles séquentiels, correction d'erreur	XGBoost
Stacking	modèles différents combinés par un méta-modèle	voir ci-dessous

1. Bagging (Bootstrap Aggregating) — déjà détaillé en section Random Forest : plusieurs modèles *du même type* entraînés en parallèle sur des échantillons bootstrap différents, puis moyennés (régression) ou votés (classification). Réduit surtout la **variance**.

2. Boosting — déjà détaillé en section XGBoost : les modèles sont entraînés *séquentiellement*, chacun corrigeant les erreurs (résidus) de l'ensemble des précédents. Réduit surtout le **biais** (et la variance).

3. Stacking (empilement) — non vu jusqu'ici. On entraîne plusieurs modèles *de types différents* (ex. Logistic Regression, Random Forest, XGBoost) sur les mêmes données ; leurs prédictions deviennent ensuite les *features d'entrée* d'un modèle final, dit *méta-modèle* (souvent une simple régression), qui apprend comment pondérer/combiner ces prédictions pour produire la prédiction finale. Exploite le fait que des familles de modèles différentes captent des patterns différents dans les données — au prix d'une complexité et d'un risque d'overfitting plus élevés (le méta-modèle doit lui aussi être entraîné avec rigueur, typiquement via cross-validation, pour éviter que ses features d'entrée ne souffrent d'un data leakage entre modèles de base et méta-modèle).

Gradient Boosting — généralisation au-delà de XGBoost

Principe déjà vu, formalisé plus largement. XGBoost est *une implémentation* (optimisée, régularisée) du principe plus général du *Gradient Boosting* : à chaque étape t , on ajoute un nouvel arbre f_t qui approxime le **gradient négatif** de la fonction de perte par rapport aux prédictions actuelles — une généralisation de "prédire le résidu" (cas particulier du MSE) à *n'importe quelle* fonction de perte différentiable (log-loss pour la classification, etc.) :

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(\mathbf{x}), \quad f_t \approx -\frac{\partial l(y, \hat{y}^{(t-1)})}{\partial \hat{y}^{(t-1)}}$$

Cette formule généralise exactement celle déjà vue pour XGBoost (section Algorithmes) — XGBoost y ajoute la régularisation $\Omega(f_t)$ et des optimisations d'implémentation (parallélisation, gestion native des valeurs manquantes) qui en font la version la plus utilisée en pratique.

Autres implémentations à connaître de nom : **LightGBM** (plus rapide sur de très gros volumes, croissance de l'arbre "par feuille" plutôt que "par niveau") et **CatBoost** (gère nativement les variables catégorielles sans encodage préalable). Pas nécessaire de maîtriser leurs

détails internes pour ce concours — savoir qu’elles existent et pourquoi (performance, volume, catégoriel) suffit à l’oral.

Question type à savoir répondre à l’oral : « *Quelle différence entre Bagging et Boosting ?* » → déjà répondue en détail dans le tableau de la section XGBoost : Bagging = arbres indépendants en parallèle, réduit la variance (Random Forest) ; Boosting = arbres séquentiels qui corrigent les erreurs précédentes, réduit le biais (XGBoost/Gradient Boosting).

4.8 Questions ouvertes

Objectif de cette sous-section. Contrairement aux questions techniques précédentes (définitions, formules), un concours de la fonction publique pose souvent des questions *ouvertes* qui n’ont pas une seule bonne réponse : on attend une réponse **structurée** (plan clair, arguments dans l’ordre) qui prouve une compréhension d’ensemble, pas juste une liste de mots-clés. Chaque question ci-dessous renvoie vers les sections déjà rédigées — c’est un exercice de synthèse, pas de nouveau contenu.

Q1 — Comment choisir un algorithme de Machine Learning ?

Structure de réponse en 4 critères :

1. **Type de problème** : classification, régression, ou clustering (section *Classification vs régression vs clustering*) — élimine déjà la moitié des algorithmes possibles.
2. **Nature et volume des données** : peu de données et features peu nombreuses → modèle simple (Linear/Logistic Regression) pour éviter l’overfitting ; beaucoup de données tabulaires avec interactions complexes → Random Forest/XGBoost ; données non structurées (image, texte) → Deep Learning (section *AI vs ML vs Deep Learning*).
3. **Interprétabilité exigée** : si le métier exige de justifier chaque décision (ex. refus d’un dossier), privilégier un modèle interprétable (Linear/Logistic Regression, Decision Tree) plutôt qu’un modèle plus performant mais "boîte noire" (question déjà traitée dans la section Random Forest : « *Pourquoi choisir Random Forest plutôt que Logistic Regression ?* »).
4. **Contraintes opérationnelles** : temps de calcul disponible, fréquence de ré-entraînement, ressources de production — un modèle légèrement moins précis mais 10 fois plus rapide peut être préférable en production.

Réponse courte à l’oral : on part toujours d’une **baseline simple** (régression linéaire/logistique), on mesure sa performance, puis on teste des modèles plus complexes (Random Forest, XGBoost) uniquement si le gain de performance justifie la perte d’interprétabilité et le coût de calcul supplémentaire.

Q2 — Comment améliorer un modèle qui sous-performe ?

Structure de réponse — distinguer d’abord la cause : un modèle sous-performant vient soit d’un problème de **données**, soit d’un problème de **modèle**.

1. **Diagnostiquer** : comparer score train vs score test. Écart important → overfitting (voir Q4) ; score faible sur train *et* test → underfitting ou données peu informatives.
2. **Agir côté données** : plus de données d’entraînement si possible ; meilleur *feature engineering* (section *Feature engineering et feature selection*) — souvent le levier le plus rentable ; vérifier l’absence de *data leakage* qui fausserait le diagnostic.
3. **Agir côté modèle** : essayer un modèle plus adapté (Q1) ; *hyperparameter tuning* rigoureux via cross-validation (sections *Cross-validation* et *Hyperparameter tuning*) ; passer à un *Ensemble Learning* (bagging/boosting/stacking, section *Gradient Boosting et Ensemble Learning*) si un seul modèle plafonne.
4. **Vérifier la métrique** : s’assurer qu’on optimise la bonne métrique métier (ex. Recall plutôt qu’Accuracy sur un dataset déséquilibré, section *Imbalanced dataset et SMOTE*) —

améliorer un modèle commence parfois par se rendre compte qu'on mesurait la mauvaise chose.

Q3 — Que faire si les classes sont déséquilibrées ?

Déjà répondue en détail dans la sous-section *Imbalanced dataset et SMOTE* — résumé pour l'oral : ne pas se fier à l'Accuracy (trompeuse sur un dataset déséquilibré) ; rééquilibrer les données (SMOTE plutôt qu'un simple oversampling, pour limiter l'overfitting) ou pondérer les classes dans le modèle (`class_weight`) ; toujours resampler *après* le split, uniquement sur le train, pour ne pas créer de data leakage ; choisir une métrique adaptée (F1, ROC-AUC, PR-AUC).

Q4 — Comment éviter l'overfitting ?

Déjà répondue en détail dans la sous-section *Questions types* (section Évaluation ML) — résumé pour l'oral : plus de données ; régularisation (L1/L2, ou γ/λ pour XGBoost) ; réduire la complexité du modèle (profondeur d'arbre, nombre de features via *feature selection*) ; cross-validation pour un choix d'hyperparamètres fiable ; early stopping. *Point à ajouter ici* : la *cross-validation* et le *hyperparameter tuning* rigoureux (sections dédiées) sont eux-mêmes des outils de *détection* de l'overfitting avant même de le corriger — un grand écart entre les K scores de validation croisée est un signal d'alerte précoce.

Q5 — Cas pratique type concours : « L'ANP souhaite prédire le temps d'attente des navires avant accostage. Quelle démarche proposez-vous ? »

Structure de réponse en 10 étapes (à dérouler à l'oral comme à l'écrit, cf. Partie F du sujet blanc) :

1. **Collecte** : rassembler les données pertinentes (historique des arrivées, tirant d'eau, type de navire, trafic du quai, conditions météo/marée).
2. **Nettoyage** : traiter les valeurs manquantes et les outliers (section *Preprocessing*).
3. **EDA (Exploratory Data Analysis)** : statistiques descriptives, corrélations, visualisations pour comprendre les données avant de modéliser.
4. **Feature engineering** : créer des variables dérivées utiles (jour de la semaine, charge du quai au moment de l'arrivée, historique de l'armateur — section *Feature engineering et feature selection*).
5. **Train/Test split** : réservé chronologiquement si les données sont temporelles (voir *Time Series Split*, section *Cross-validation*) pour éviter une fuite temporelle.
6. **Choix du modèle** : régression (le target — temps d'attente — est continu, section *Classification vs régression vs clustering*) ; baseline Linear Regression, puis Random Forest/XGBoost si les relations sont non linéaires (Q1).
7. **Évaluation** : MAE/RMSE/ R^2 (section *Régression — Métriques*), avec cross-validation pour une estimation fiable.
8. **Déploiement** : exposer le modèle via une API ou l'intégrer au système d'information portuaire existant.
9. **Monitoring** : suivre la performance en production dans le temps (dérive des données/*data drift* si le trafic ou les procédures changent), déclencher un ré-entraînement si la performance se dégrade.
10. **Dashboard** : restituer les prédictions aux équipes opérationnelles (ex. Power BI/Tableau) pour une aide à la décision concrète — anticiper l'allocation des quais, réduire les temps d'attente.